

Data Wrangling

Transform Raw Messy Data Into Analysis-Ready Gold

■ DATA TRANSFORMATION

■ LEARNING OUTCOME

By the end of this PDF you will reshape, merge, clean, and restructure any messy real-world dataset into a clean, consistent, analysis-ready format using Pandas.

■ Skills You Will Gain

- Merge and join multiple DataFrames
- Reshape data: melt, pivot, stack/unstack
- Handle messy text: strip, split, replace, regex
- Parse and engineer datetime features
- Handle outliers systematically
- Build a reusable data wrangling pipeline

■ Data Wrangling consumes 60-80% of a data scientist's time. Master it and you'll be faster than 90% of your peers.

SECTION 1

Merging & Joining DataFrames

Join Type	Returns	Code
inner	Only matching rows in BOTH	<code>pd.merge(a,b,on='id',how='inner')</code>
left	All of left + matching right	<code>pd.merge(a,b,on='id',how='left')</code>
right	All of right + matching left	<code>pd.merge(a,b,on='id',how='right')</code>
outer	All rows from both	<code>pd.merge(a,b,on='id',how='outer')</code>
concat	Stack rows vertically	<code>pd.concat([df1,df2],ignore_index=True)</code>
combine_first	Fill NaNs from second df	<code>df1.combine_first(df2)</code>

SECTION 2

Reshaping Data

```
# MELT – wide to long (unpivot) # Before: Name | Math | Science | English # After: Name | Subject | Score
melted = df.melt(id_vars=['Name'], var_name='Subject', value_name='Score') # PIVOT – long to wide
pivoted = df.pivot(index='Name', columns='Subject', values='Score') # PIVOT TABLE – like Excel pivot with aggregation
pt = df.pivot_table(values='Revenue', index='Month', columns='Region', aggfunc='sum', fill_value=0) # STACK/UNSTACK – move column level to row index
stacked = df.stack() # columns → rows
unstacked = df.unstack() # rows → columns
```

SECTION 3

String / Text Wrangling

```
# All string operations via .str accessor
df['Name'] = df['Name'].str.strip() # remove whitespace
df['Name'] = df['Name'].str.lower() # lowercase
df['Email'] = df['Email'].str.replace('@gmail.com','') # remove suffix
# Split a column
df[['First','Last']] = df['FullName'].str.split(' ', expand=True) # Extract with regex
df['ZipCode'] = df['Address'].str.extract(r'(\d{6})') # Check contains
df[df['Product'].str.contains('iPhone', na=False)] # Replace multiple values
df['Gender'] = df['Gender'].replace({'M':'Male','F':'Female','m':'Male'})
```

SECTION 4

DateTime Engineering

```
df['Date'] = pd.to_datetime(df['Date']) # parse string to datetime # Extract components
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Day'] = df['Date'].dt.day
df['DayOfWeek'] = df['Date'].dt.dayofweek # 0=Mon, 6=Sun
df['Quarter'] = df['Date'].dt.quarter # Time differences
df['Age_Days'] = (pd.Timestamp.today() - df['JoinDate']).dt.days
df['Age_Years'] = df['Age_Days'] / 365.25 # Filter by date range
df_q1 = df[(df['Date'] >= '2024-01-01') & (df['Date'] < '2024-04-01')] # Resample time series (monthly totals)
```

```
df.set_index('Date').resample('M')['Revenue'].sum()
```

SECTION 5

Handling Outliers

Method	Code	Best For
IQR Method	<code>Q1=df.quantile(.25); Q3=df.quantile(.75); IQR=Q3-Q1 df[(df>=Q1-1.5*IQR)&(df<=Q3+1.5*IQR)]</code>	Most numeric distributions
Z-Score	<code>from scipy.stats import zscore df[abs(zscore(df['col']))<3]</code>	Normally distributed data
Percentile Clip	<code>df['col'].clip(lower=df['col'].quantile(.01), upper=df['col'].quantile(.99))</code>	When you want to cap, not remove
Log Transform	<code>np.log1p(df['col'])</code>	Right-skewed data (income, price)

SECTION 6

Building a Wrangling Pipeline

```
def wrangle(filepath): df = pd.read_csv(filepath) # 1. Rename columns df.columns =  
df.columns.str.lower().str.replace(' ', '_') # 2. Drop useless columns  
df.drop(columns=['unnamed:_0','index'], errors='ignore', inplace=True) # 3. Handle missing values  
df['age'].fillna(df['age'].median(), inplace=True) df.dropna(subset=['target'], inplace=True) # 4.  
Fix types df['date'] = pd.to_datetime(df['date']) df['price'] = df['price'].astype(float) # 5.  
Remove duplicates df.drop_duplicates(inplace=True) # 6. Feature engineering df['year'] =  
df['date'].dt.year return df clean_df = wrangle('raw_data.csv')
```

■ PRACTICE QUESTIONS

Q1. What is the difference between `pd.merge()` and `pd.concat()`?

Answer: `merge()` joins DataFrames horizontally based on a common key column. `concat()` stacks DataFrames vertically (adds rows).

Q2. When would you use `melt()` instead of `pivot()`?

Answer: `melt()` converts wide format (one column per category) to long format (one row per observation) — useful for tidy data and visualization.

Q3. How do you extract the month from a datetime column 'OrderDate'?

Answer: `df['Month'] = df['OrderDate'].dt.month` — after converting to datetime with `pd.to_datetime()`.

Q4. Why is log transformation useful for skewed data?

Answer: Log compresses extreme values, reducing skewness and making distributions more normal — required by many ML models.

Q5. How would you standardize inconsistent gender values like 'M','male','MALE' to one value?

Answer: `df['Gender'] = df['Gender'].str.lower().str.strip().replace({'m':'male','f':'female'})`

■ Data Wrangling Cheat Sheet	
<code>pd.merge(a,b,on='key')</code>	Join two DataFrames
<code>pd.concat([a,b])</code>	Stack DataFrames vertically
<code>df.melt(id_vars,var_name)</code>	Wide → Long format

<code>df.pivot(index, columns, values)</code>	Long → Wide format
<code>df['col'].str.strip()</code>	Remove whitespace
<code>df['col'].str.split(' ', expand=True)</code>	Split text into columns
<code>df['col'].str.extract(r'pattern')</code>	Regex extraction
<code>pd.to_datetime(df['col'])</code>	Parse dates
<code>df['date'].dt.year/month/day</code>	Extract date parts
<code>df['col'].clip(lower, upper)</code>	Cap outliers
<code>np.log1p(df['col'])</code>	Log transform skewed
<code>df.replace({'old': 'new'})</code>	Bulk value replacement